



PDF Download
3555353.pdf
20 December 2025
Total Citations: 5
Total Downloads:
2699

Latest updates: <https://dl.acm.org/doi/10.1145/3555353>

RESEARCH-ARTICLE

Cache-oblivious Hilbert Curve-based Blocking Scheme for Matrix Transposition

JOÃO NUNO ALVES, Institute of Systems and Computer Engineering:
Research and Development in Lisbon, Lisbon, Lisbon, Portugal

LUÍS MANUEL S RUSSO, Institute of Systems and Computer
Engineering: Research and Development in Lisbon, Lisbon, Lisbon, Portugal

ALEXANDRE P FRANCISCO, Institute of Systems and Computer
Engineering: Research and Development in Lisbon, Lisbon, Lisbon, Portugal

Open Access Support provided by:

Institute of Systems and Computer Engineering: Research and
Development in Lisbon

Published: 19 December 2022

Online AM: 09 August 2022

Accepted: 27 June 2022

Revised: 25 May 2022

Received: 07 January 2021

[Citation in BibTeX format](#)

Cache-oblivious Hilbert Curve-based Blocking Scheme for Matrix Transposition

JOÃO NUNO FERREIRA ALVES, LUÍS MANUEL SILVEIRA RUSSO, and
ALEXANDRE FRANCISCO, INESC-ID, Portugal

This article presents a fast SIMD Hilbert space-filling curve generator, which supports a new cache-oblivious blocking-scheme technique applied to the out-of-place transposition of general matrices. Matrix operations found in high performance computing libraries are usually parameterized based on host microprocessor specifications to minimize data movement within the different levels of memory hierarchy. The performance of cache-oblivious algorithms does not rely on such parameterizations. This type of algorithm provides an elegant and portable solution to address the lack of standardization in modern-day processors. Our solution consists in an iterative blocking scheme that takes advantage of the locality-preserving properties of Hilbert space-filling curves to minimize data movement in any memory hierarchy. This scheme traverses the input matrix, in $O(nm)$ time and space, improving the behavior of matrix algorithms that inherently present poor memory locality. The application of this technique to the problem of out-of-place matrix transposition achieved competitive results when compared to state-of-the-art approaches. The performance of our solution surpassed Intel MKL version after employing standard software prefetching techniques.

CCS Concepts: • **Mathematics of computing** → **Mathematical software performance**; • **Theory of computation** → **Algorithm design techniques**; *Abstract machines*;

Additional Key Words and Phrases: Cache-oblivious algorithms, out-of-place transposition, Hilbert space-filling curve, dense matrix

ACM Reference format:

João Nuno Ferreira Alves, Luís Manuel Silveira Russo, and Alexandre Francisco. 2022. Cache-oblivious Hilbert Curve-based Blocking Scheme for Matrix Transposition. *ACM Trans. Math. Softw.* 48, 4, Article 37 (December 2022), 28 pages.

<https://doi.org/10.1145/3555353>

1 INTRODUCTION

Underneath most fields of computer science one will find matrix operations to be one of the inevitable primitive structures whose operations allow the computation of other more complex

João Nuno Ferreira Alves with Research Group Scientific Computing, Faculty of Computer Science, University of Vienna. J. N. F. Alves, L. M. S. Russo, and A. Francisco with Instituto Superior Técnico, University of Lisbon.

The work reported in this article was supported by national funds through Fundação para a Ciência e Tecnologia (FCT) with references UIDB/50021/2020, PTDC/CCI-BIO/29676/2017, LA/P/0078/2020 and in part by the Austrian Science Fund (FWF) project P 33884.

Authors' address: J. N. F. Alves, L. M. S. Russo, and A. Francisco, INESC-ID, R. Alves Redol 9, Lisbon, Portugal; emails: joao.ferreira.alves@ist.utl.pt, luis.russo@tecnico.ulisboa.pt, aplf@ist.utl.pt.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2022 Copyright held by the owner/author(s).

0098-3500/2022/12-ART37

<https://doi.org/10.1145/3555353>

algorithms. The most severe bottleneck when dealing with large matrices is the overhead imposed by fetching matrix data from main memory to cache. To achieve an efficient implementation, the programmer should use cache-friendly solutions, which exploits cache temporal and spatial locality. Several scientific computation libraries implement cache-friendly solutions for matrix operations with near optimum runtime [12, 18, 24]. These libraries make efficient use of cache by fine-tuning a set of parameters whose values depend on cache specifications, such as the cache line size and the overall memory hierarchy. These solutions are not portable, since cache specifications are not standardized. Each machine architecture requires its own solution.

1.1 Cache-oblivious Algorithms

There are, however, cache-efficient algorithms whose performance is independent from cache specifications. These algorithms are known as cache-oblivious [11]. This set of algorithms usually procures to obtain high cache hit-rates through recursive divide-and-conquer methods. At a given point in time each sub-problem will fit in cache. On modern CPUs, these solutions are rarely more efficient than their cache-aware counterparts, due to their recursive nature. One way to obtain cache-oblivious solutions is to make use of the inherent locality preservation properties of space-filling curves, such as the Hilbert, Peano, and Z-order curves. The application of space-filling curves to optimize dense matrix operations is an extensively studied field [2, 3, 16]. Both Hilbert [13] and Z-order [20] curves are defined as a curve able to traverse any $2^l \times 2^l$ grid, where $l \in \mathbb{N}_0$, visiting each of its entries exactly once. This ensures the existence of the bijective functions $\mathcal{H}^{-1}(h)$ and $\mathcal{Z}^{-1}(z)$ that map a curve point h or z into a two-dimensional point (i, j) of the Hilbert and Z-order curve, respectively. It is important to note that most of the methods used to implement these curves are also recursive, incurring the same inefficiencies as the recursive divide-and-conquer approaches.

In a recent work, Böhm et al. [5] presented FUR-Hilbert, a locality-preserving nested-loop that enforces a grid traversal scheme based on a Hilbert space-filling curve. Their approach uses small pre-processed traversal paths, called nano-programs, that are then tessellated based on a given iteration of a Hilbert curve. Their solution presents a constant overhead per loop iteration and is completely iterative, resulting in an efficient strategy that can be easily optimized by the compiler. The locality-preserving characteristics of this loop are dependent on the original matrix traversal scheme used in the host algorithm. Due to nature of Hilbert space-filling curves, this loop can only be used in algorithms that do not present any type of data dependencies, such as matrix multiplication.

1.2 Cache-oblivious Out-of-place Transposition

In contrast to other dense matrix operations, the cache-oblivious out-of-place transposition of rectangular matrices did not seem to receive as much attention from the scientific community. Chatterjee et al. compared different approaches to compute the in-place transposition of square matrices [7]. These approaches included cache-oblivious strategies based on divide-and-conquer or space-filling curves. The approach that presented the best execution time was in fact a strategy that leverages the locality preserving properties of a Z-order curve. While their solution is not directly translatable to this problem, it suggests that the out-of-place transposition of rectangular matrices can be improved by using similar strategies.

The out-of-place transposition problem of a rectangular $n \times m$ matrix M is usually defined as writing the rows of M as the columns of a $m \times n$ matrix T , i.e., $\forall_{i,j} T_{j,i} := M_{i,j}$, where $0 \leq i < n$ and $0 \leq j < m$. This problem is typically solved using a RAM-model approach based on the following nested-loop:

```

for  $i := 0; i < n; i := i + 1$  do
  for  $j := 0; j < m; j := j + 1$  do
     $T_{j,i} := M_{i,j};$ 

```

This type of solution presents a simpler nested-loop that incurs in a minimal amount of arithmetic instructions but disregards the behavior of the memory hierarchy. This loop implies that either M or T , both allocated in row/column-major order layout, will be scanned using a column/row-major order traversal. This results in an unnecessary amount of capacity and conflict cache misses, and even page faults, for sufficiently large matrices. The transposition of matrices is a memory-bound problem. For this type of problem, it is usually beneficial to favor improved memory access patterns over reducing the number of arithmetic instructions. The performance degrading effects of the standard out-of-place transposition algorithm can be intuitively understood by comparing this approach with different access patterns in a simplistic cache model. Assume our cache has the following characteristics:

- It is composed by four lines, each able to hold two single-precision values;
- It is fully associative, meaning conflict misses do not occur;
- It employs the Least-recently-used replacement policy;
- It also employs the Write-allocate policy, i.e., read before write.

Let M and T be 4×4 single-precision matrices allocated in memory using the row-major layout. M corresponds to the input matrix of problem, while T corresponds to the output matrix. The standard transposition approach to compute the out-of-place transpose of M is depicted by the following nested-loop:

```

for  $i := 0; i < 4; i := i + 1$  do
  for  $j := 0; j < 4; j := j + 1$  do
     $T_{j,i} := M_{i,j};$ 

```

Running this strategy on the previously described cache results in a total of 24 cache misses, as illustrated in Figures 1(a) and 1(d). One might argue that the resulting amount of cache misses in T is a direct consequence of our small cache size. This is not the case. Any matrix that is stored in row-major order and later scanned using column-major order will result in poor spatial locality preservation. Whenever the number of matrix columns is larger than cache line size and the number of matrix rows is greater than the number of cache lines, it is likely that memory accesses to the output matrix will result in a cache-miss.

To avoid traversing the entries of T using column-major order, one can use the scheme displayed by Figures 1(b) or 1(c), which make use of the Hilbert or Z-order inverse function, represented by $\mathcal{H}^{-1}$ or $\mathcal{Z}^{-1}$, to impose a Hilbert or Z-order space-filling curve based traversal over the input matrix M , as:

<pre> for $h := 0; h < 15; h := h + 1$ do $(i, j) := \mathcal{H}^{-1}(h);$ $T_{j,i} := M_{i,j};$ </pre>	<pre> for $z := 0; z < 15; z := z + 1$ do $(i, j) := \mathcal{Z}^{-1}(z);$ $T_{j,i} := M_{i,j};$ </pre>
---	---

Both traversal schemes ensure that the output matrix T will be scanned by a reflection of the previous Hilbert or Z-order space-filling curve, as shown by Figures 1(e) and 1(f), respectively. Simulating these strategies on the cache defined above only results 16 cache misses. Typically, the amount of time required to read data from main memory is around 70 ns, whereas loading data from cache only incurs in 2 ns, as stated by Arroz et al. [1]. Disregarding any cost other than the

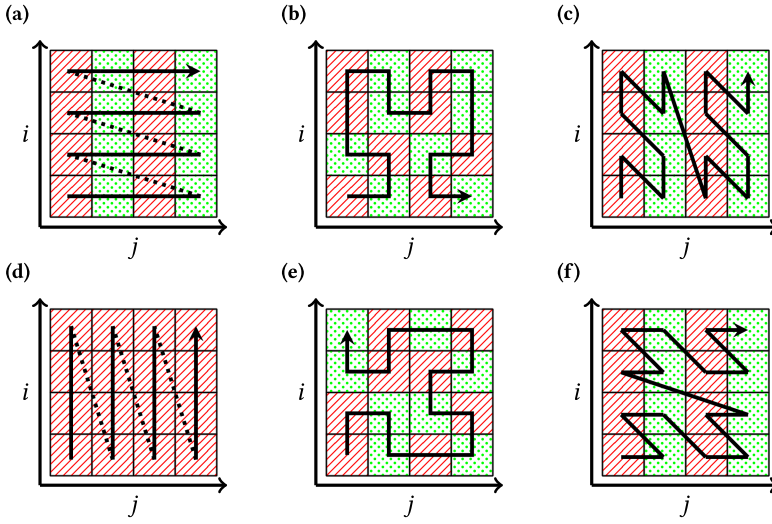


Fig. 1. Traversal scheme of matrices (a, b, c) M and (d, e, f) T , based on a row-major order, Hilbert, and Z-order space-filling curve. Cache-Hits are represented by entries filled with green dots, while cache misses are represented by entries filled by red diagonal skipped lines.

load time from cache (cache-hit) and main memory (cache-miss) the naive approach presents an overhead of $70 \times 24 + 2 \times 8 = 1,696$ ns, while the other space-filling curve-based approaches only take $70 \times 16 + 2 \times 16 = 1,152$ ns. If one neglects the arithmetic cost of these operations, then it is possible to conclude that the latter strategies present a performance speedup close to 33%.

1.3 Contributions and Document Organization

This document presents two major contributions:

- (1) In Section 3, we introduce a linear Hilbert space-filling curve generator, which it is not only iterative, but also free of data dependencies, which allows the implementation of a solution that mostly uses SIMD instructions and can be efficiently optimized by most compilers. Our generator requires the curve to be stored in memory, meaning it is possible to amortize the cost of building this curve, whenever two or more generations of the same curve are required. The proof of correctness of this algorithm is based on the Lindenmayer-System defined in Section 2.
- (2) In Section 4, we build upon FUR-Hilbert to develop an alternative blocking scheme able to leverage the locality-preserving properties of the Hilbert and Z-order space-filling curves on rectangular grids. The general idea of this strategy is to partition the input matrix into $2^l \times 2^l$ smaller sub-matrices. The order of traversal of these sub-matrices is then defined by one or more Hilbert or Z-order space-filling curve, as illustrated in Figures 2(a) and 2(b). In contrast to the original approach, our blocking scheme allows the use of standard prefetching techniques that positively impact the performance of the out-of-place transposition problem.

In Section 5, we compare our SIMD Hilbert space-filling curve generator against other solutions that are known to compute a Hilbert curve in linear time, with regard to the number of curve elements. Still in this section, we present an extensive analysis of our blocking scheme in the context of the out-of-place transposition of rectangular matrices and compare our cache-oblivious strategies against the current state-of-the art, including cache-aware solutions. The

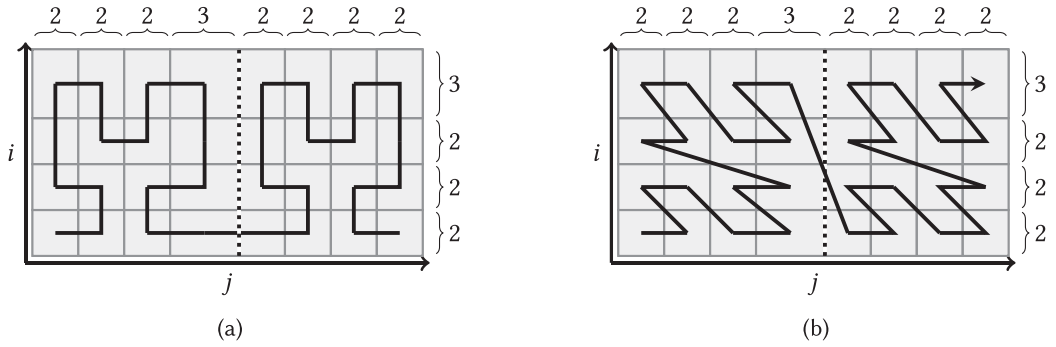


Fig. 2. Blocking-scheme example of two 9×17 grids based on (a) a Hilbert space-filling curve and (b) a Z-order curve. Each block, sub-matrix, is scanned using row-major order.

implementation, in the C programming language, of our cache-oblivious approaches can be found at: <https://github.com/JoaoAlves95/HPC-Cache-Oblivious-Transposition>.

2 KNOWN METHODS TO COMPUTE A HILBERT SPACE-FILLING CURVE

There are several classical methods that compute iteration l of a Hilbert space-filling curve. However, such algorithms often present a time cost of $O(\log(N))$ per entry [6, 8, 15]. The time complexity of these algorithms presents a significant overhead when compared to traditional for-nested loops that enumerate all entries of a matrix in $O(N)$, i.e., $O(1)$ per entry.

The Lindenmayer-System [22] is a parallel rewriting system, based on a context-free grammar [9], capable of generating any iteration of a Hilbert space-filling curve in linear time $O(N)$. This system receives as input the desired level of expansion l and returns a sequence of symbols in $\{\rightarrow, \uparrow, \leftarrow, \downarrow\}$ that describe the traversal performed by iteration l of a given Hilbert curve. Each of these symbols describes a unit-step along the horizontal or vertical coordinate axis, where

$$\rightarrow \text{ corresponds to } j := j + 1, \quad (1)$$

$$\leftarrow \text{ corresponds to } j := j - 1, \quad (2)$$

$$\uparrow \text{ corresponds to } i := i + 1, \quad (3)$$

$$\downarrow \text{ corresponds to } i := i - 1. \quad (4)$$

To keep this document self-contained, we adapted the characterization presented by Bader [2] to define this system, as seen in Definition 1. This definition will be useful in Section 3 to prove the correction of our strategy.

Definition 1. Lindenmayer-System for Hilbert Space-filling Curves.

The standard Lindenmayer-System designed to generate Hilbert space-filling curves is composed by:

- (1) A set of non-terminal symbols: $\{H, A, B, C\}$. Each non-terminal represents one of the four basic patterns as depicted in Figure 3. For our purposes, we will assume the axiom, i.e., the starting symbol, is either H or A .
- (2) A set of terminal symbols: $\{\rightarrow, \uparrow, \leftarrow, \downarrow, \varepsilon\}$. Each arrow symbol represents a unit step onto the direction it depicts. The terminal ε corresponds to the empty symbol, and it is used to remove any non-terminal from the last string generated by our grammar, resulting in a string containing only terminal symbols.

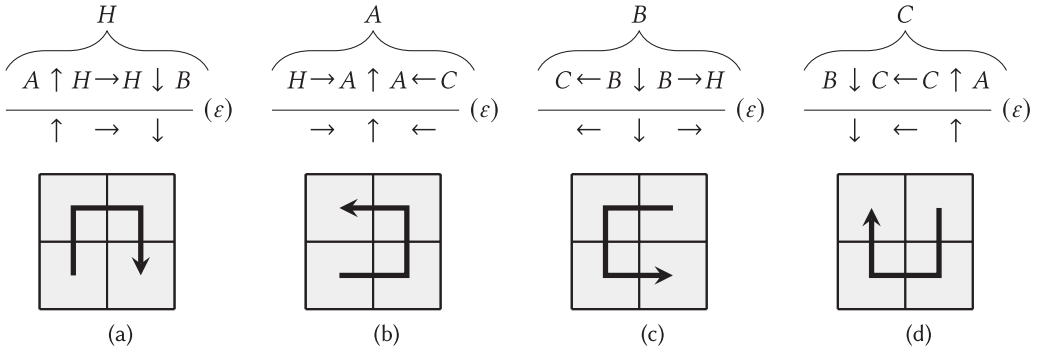


Fig. 3. First expansion step and subsequent elimination of non-terminal symbols of a Lindenmayer-System with axiom (a) H , (a) A , (c) B , and (d) C . The sequence of non-terminal terminal symbols present in each of the first expansion steps characterize the four primitive patterns (iteration $l = 1$) of a Hilbert space-filling curve.

(3) A set of production rules:

$$H \xrightarrow{\text{Produces}} \epsilon \mid A \uparrow H \rightarrow H \downarrow B,$$

$$A \xrightarrow{\text{Produces}} \epsilon \mid H \rightarrow A \uparrow A \leftarrow C,$$

$$B \xrightarrow{\text{Produces}} \epsilon \mid C \leftarrow B \downarrow B \rightarrow H,$$

$$C \xrightarrow{\text{Produces}} \epsilon \mid B \downarrow C \leftarrow C \uparrow A.$$

- (4) Additional expansion rule 1: To construct a string of this grammar that corresponds to an iteration of a Hilbert curve, all non-terminal symbols present in the current expansion must be expanded at once.
- (5) Additional expansion rule 2: We are only allowed to expand a non-terminal into ϵ once we reach the desired level of expansion of this grammar. At this point, all non-terminal symbols must be replaced by ϵ .

Figure 4 illustrates the construction of the second iteration $l = 2$ of a Hilbert space-filling curve through a Lindenmayer-System with axiom H . At this point, we have all the required information to generate any iteration of a Hilbert curve in linear time, i.e., $O(1)$ cost per grid entry, and memory cost of $O(\log_4(N))$ in the function stack. This approach looks like a promising candidate. Nevertheless, its recursive nature does not allow modern compilers to efficiently generate optimized code as pointed out by Böhm et al. [5]. To address this problem, these authors developed the first non-recursive Lindenmayer-System by mimicking the stack behavior of the recursive Lindenmayer-System.

Their approach is quite efficient with $O(N)$ time and constant memory complexity. However, it presents data dependencies that restrict the use of SIMD instructions, implying faster solutions are possible if one is able to resolve or minize these dependencies.

3 SIMD HILBERT SPACE-FILLING CURVE GENERATOR

To further reduce the overhead present in the algorithms of Section 2, our solution must be completely iterative and run in linear time, resulting in code that can be easily extent with SIMD instructions or auto-vectorized by modern compilers. Such a solution is possible through the implementation of a linear and incremental Hilbert space-filling curve generator, based on bit-wise exclusive-or operations, resulting in an approach free of data dependencies.

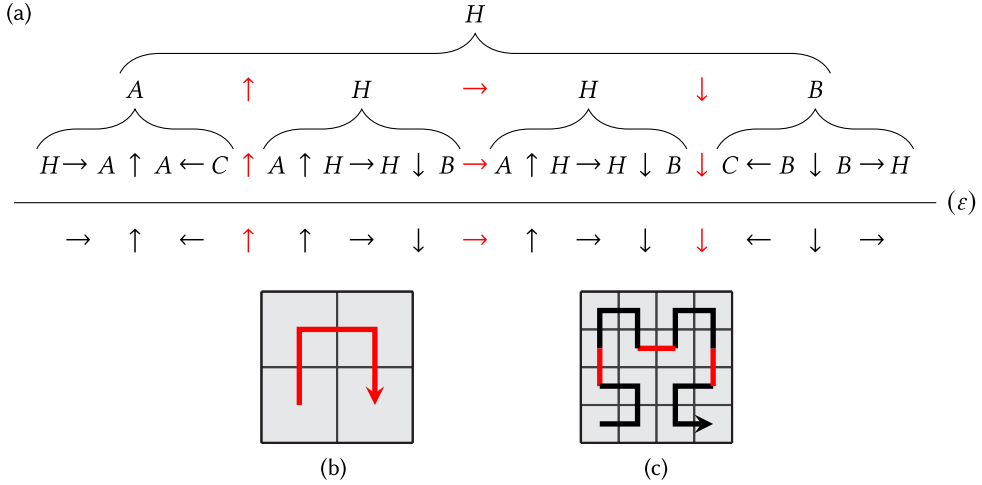


Fig. 4. (a) illustrates two expansion steps of a Lindenmayer-System produced by axiom H , followed by the elimination of the non-terminal symbols of this grammar. The terminal symbols present in the first level of expansion characterize the directions present in the first iteration of the Hilbert curve, as depicted in (b). The terminal symbols present in the second level of expansion characterize the second iteration of the same curve, as depicted in (c).

To get a better grasp of the underlying concepts that allow the construction of such generator, it is important to understand the encoding used to represent a Hilbert space-filling curve and how to convert the current direction d into unit-step within our grid. So far, a Hilbert space-filling curve has been described as a sequence of directions, where each direction is represented by one of the terminal symbols in $\{\rightarrow, \uparrow, \leftarrow, \downarrow\}$. Using this notation, the sequence of directions (curve) that describe the second iteration $l = 2$ of a Hilbert space-filling curve produced by axiom H would be represented as

$$\text{curve} = \rightarrow \uparrow \leftarrow \uparrow \uparrow \rightarrow \downarrow \rightarrow \uparrow \rightarrow \downarrow \downarrow \leftarrow \downarrow \rightarrow .$$

Naturally, this encoding is not suitable for future arithmetic manipulation and analysis of this curve. To address this issue, Böhm et al. [5] proposed an alternative encoding that maps the k th terminal symbol in $\{\rightarrow, \uparrow, \leftarrow, \downarrow\}$ to the k th integer in $\{0, 1, 2, 3\}$. This encoding provides means to mathematically represent a curve as a single base-4 number, where each direction is represented by a single digit and the sequence of directions is read from left to right. The application of this encoding to the previous sequence of directions would result in

$$\text{curve} = \underline{0\ 1\ 2\ 1\ 1\ 0\ 3\ 0\ 1\ 0\ 3\ 3\ 2\ 3\ 0}_4,$$

where $\underline{\quad}_4$ denotes that the curve is represented by a base-4 number and the order by which the directions must be processed. As previously stated, this encoding also provides means to convert a given direction $d \in \{\rightarrow_4, \uparrow_4, \downarrow_4, \leftarrow_4\}$ into a unit-step that respects Equations (1) to (4) by resorting to the remainder operator ($\%1$), as:

$$\begin{aligned} j &:= j - ((d - 1)\%2); \\ i &:= i - ((d - 2)\%2); \end{aligned}$$

¹In C, the remainder operator is equivalent to the modulo operation if the left-hand-side term is positive. In case the left-hand-side term is negative, the result of the remainder operator is equivalent to computing the symmetric of the modulo operation.

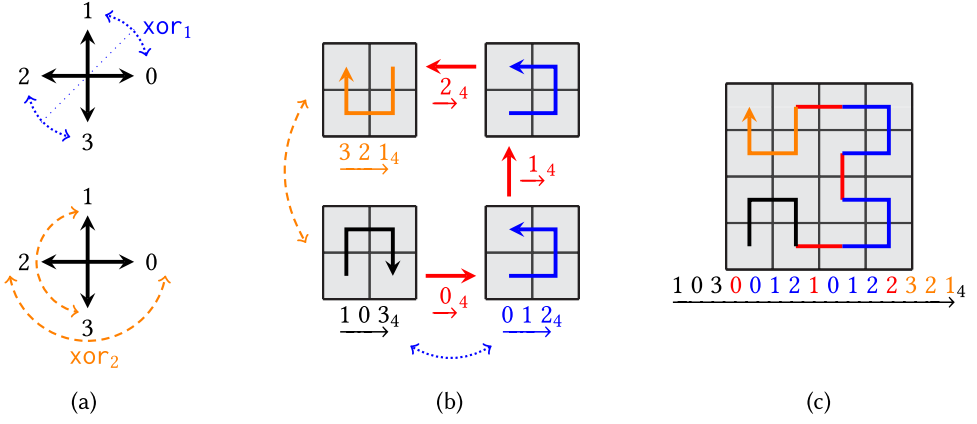


Fig. 5. (a) depicts the effect of applying bit-wise exclusive-or operations xor_1 , reflection over $y = x$ axis, and xor_2 , a 180° rotation, to a single direction. (b) illustrates the results of the same operations on a Hilbert curve with $l = 1$. (c) depicts the concatenation process of the previous computed quadrants to obtain the iteration $l = 2$ of this curve.

3.1 Computing Curve Quadrants

Figure 5 suggests that the second and third quadrant of iteration l can be obtained by reflecting the first quadrant of this curve along the $y = x$ line, while the fourth quadrant of iteration l can be computed by rotating the first quadrant of this curve by 180° . As illustrated by this figure, it seems to be possible to encode the reflection and rotation operations as an application of a bit-wise exclusive-or by 1 (xor_1) and a bit-wise exclusive-or by 2 (xor_2) to each direction contained by the first quadrant of this curve:

$$\text{xor}_1(\underline{0\ 1\ 2}_4) = \underline{1\ 0\ 3}_4 = \uparrow \rightarrow \downarrow, \quad (5)$$

$$\text{xor}_2(\underline{0\ 1\ 2}_4) = \underline{2\ 3\ 0}_4 = \leftarrow \downarrow \rightarrow. \quad (6)$$

This intuition is formalized by Propositions 2 and 3.

PROPOSITION 2. *The application of xor_1 to the first quadrant of a given Hilbert space-filling curve results in the second and third quadrant of the same iteration.*

PROOF. Assume our curve is generated by the Lindenmayer-System characterized by Definition 1 and that the axiom of this system is either the non-terminal A or H . The k th quadrant of a given Hilbert curve is characterized by the k th non-terminal symbol present in the string obtained by expanding any of the axioms once, i.e., the string obtained at level $l = 1$.

$$H \xrightarrow{\text{Produces}} A \uparrow H \rightarrow H \downarrow B \quad (7)$$

$$A \xrightarrow{\text{Produces}} H \rightarrow A \uparrow A \leftarrow C \quad (8)$$

This fact allows us to formally restate Proposition 2 as $\text{xor}_1(H_l) = A_l \wedge \text{xor}_1(B_l) = C_l$, where $l \in \mathbb{N}^+$ represents the number of expansion steps. Proposition 2 will be now proven by structural induction.

Inductive Hypothesis: $\text{xor}_1(H_l) = A_l \wedge \text{xor}_1(B_l) = C_l$, where H_l , A_l , B_l and C_l represent the curve produced by expanding this non-terminal l times, for $l \in \mathbb{N}^+$.

Base Case: Expanding exactly once the non-terminal symbols present in the first expansion step, $l = 1$, of each axiom results in

$$\begin{aligned} H_1 &\xrightarrow{\text{Produces}} \uparrow \rightarrow \downarrow = \underline{1\ 0\ 3}_4, \\ A_1 &\xrightarrow{\text{Produces}} \rightarrow \uparrow \leftarrow = \underline{0\ 1\ 2}_4, \\ B_1 &\xrightarrow{\text{Produces}} \leftarrow \downarrow \rightarrow = \underline{2\ 3\ 0}_4, \\ C_1 &\xrightarrow{\text{Produces}} \downarrow \leftarrow \uparrow = \underline{3\ 2\ 1}_4. \end{aligned}$$

Thus, the application of xor_1 to the first step of expansion of non-terminals H and B results in the sequences of directions presented in Equations (9) and (10), respectively. This is

$$\text{xor}_1(H_1) = \text{xor}_1(\underline{1\ 0\ 3}_4) = \underline{0\ 1\ 2}_4 = A_1, \quad (9)$$

and

$$\text{xor}_1(B_1) = \text{xor}_1(\underline{2\ 3\ 0}_4) = \underline{3\ 2\ 1}_4 = C_1. \quad (10)$$

It is possible to observe that the base case holds. The sequence of directions produced by one expansion step of non-terminal symbol A_1 is equal to the one produced by $\text{xor}_1(H_1)$, while the sequence of directions produced by one expansion step of the non-terminal C_1 is equal to the one generated by $\text{xor}_1(B_1)$.

Inductive Step: The application of xor_1 to the sequence of symbols produced by non-terminal symbols H and B results in equations

$$\text{xor}_1(H_l) = \text{xor}_1(A_{l-1} \uparrow H_{l-1} \rightarrow H_{l-1} \downarrow B_{l-1}), \quad (11)$$

and

$$\text{xor}_1(B_l) = \text{xor}_1(C_{l-1} \leftarrow B_{l-1} \downarrow B_{l-1} \rightarrow H_{l-1}). \quad (12)$$

Note that xor_1 can be applied element-wise over the sequence, i.e., the application of this function to a sequence of directions or to each individual element of this sequence holds the same result. Thus, it is possible to further simplify Equations (11) and (12) as Equations (13) and (14), respectively,

$$\text{xor}_1(H_l) = \text{xor}_1(A_{l-1}) \text{ xor}_1(\uparrow) \text{ xor}_1(H_{l-1}) \text{ xor}_1(\rightarrow) \text{ xor}_1(H_{l-1}) \text{ xor}_1(\downarrow) \text{ xor}_1(B_{l-1}), \quad (13)$$

and

$$\text{xor}_1(B_l) = \text{xor}_1(C_{l-1}) \text{ xor}_1(\leftarrow) \text{ xor}_1(B_{l-1}) \text{ xor}_1(\downarrow) \text{ xor}_1(B_{l-1}) \text{ xor}_1(\rightarrow) \text{ xor}_1(H_{l-1}). \quad (14)$$

Remind that the terminal symbols ($\rightarrow$, $\uparrow$, $\leftarrow$, $\downarrow$) are encoded as (0, 1, 2, 3). This allows us to rewrite the applications of xor_1 to the terminal symbols present in Equations (13) and (14) as

$$\text{xor}_1(H_l) = \text{xor}_1(A_{l-1}) \rightarrow \text{xor}_1(H_{l-1}) \uparrow \text{ xor}_1(H_{l-1}) \leftarrow \text{ xor}_1(B_{l-1}), \quad (15)$$

and

$$\text{xor}_1(B_l) = \text{xor}_1(C_{l-1}) \downarrow \text{ xor}_1(B_{l-1}) \leftarrow \text{ xor}_1(B_{l-1}) \uparrow \text{ xor}_1(H_{l-1}). \quad (16)$$

Our inductive hypothesis implies that $\text{xor}_1(H_{l-1}) = A_{l-1}$ and $\text{xor}_1(B_{l-1}) = C_{l-1}$ for $l > 1$. Since xor_1 is an idempotent function it is also implied that $\text{xor}_1(A_{l-1}) = H_{l-1}$ and $\text{xor}_1(C_{l-1}) = B_{l-1}$. By our inductive hypothesis, it is possible to simplify once again Equations (15) and (16) as

$$\text{xor}_1(H_l) = H_{l-1} \rightarrow A_{l-1} \uparrow A_{l-1} \leftarrow C_{l-1} = A_l, \quad (17)$$

and

$$\text{xor}_1(B_l) = B_{l-1} \downarrow C_{l-1} \leftarrow C_{l-1} \uparrow A_{l-1} = C_l. \quad (18)$$

Note that the non-terminal symbols present in the first expansion step of $\text{xor}_1(H_l)$ and $\text{xor}_1(B_l)$ are the same as the non-terminals present in the first expansion step of A_l and C_l , respectively. This concludes the proof of Proposition 2. $\square$

PROPOSITION 3. *The application of xor_2 to the first quadrant of a given iteration of a Hilbert space-filling curve results in the fourth quadrant of the same iteration.*

Similarly to the proof of Proposition 2, assume that our curve is generated by the Lindenmayer-System characterized by Definition 1 and that the k th quadrant of a given Hilbert curve is characterized by the k th non-terminal symbol present in the string obtained by expanding any of the axioms once:

$$H \xrightarrow{\text{Produces}} A \uparrow H \rightarrow H \downarrow B, \quad (19)$$

$$A \xrightarrow{\text{Produces}} H \rightarrow A \uparrow A \leftarrow C. \quad (20)$$

The statement of Proposition 3 is equivalent to $\text{xor}_2(A_l) = B_l \wedge \text{xor}_2(H_l) = C_l$, where $l \in \mathbb{N}^+$ represents the number of expansion steps. Proposition 3 can be proven by structural induction following the strategy used in Proposition 2; to keep this document brief, we opted to omit this proof.

3.2 Connecting Curve Quadrants

At this point, we have seen how to generate the three last quadrants of a given iteration of a Hilbert space-filling curve. To complete the generation of this curve, one needs to connect the previously computed quadrants. The directions used to connect these quadrants can be found in the first step of expansion of this grammar. From Propositions 2 and 3 follows that the next step of expansion, $l + 1$, can be generated based on the current expansion step of H_l and A_l as

$$H_{l+1} = \text{xor}_1(H_l) \uparrow H_l \rightarrow H_l \downarrow \text{xor}_2(\text{xor}_1(H_l)), \quad (21)$$

and

$$A_{l+1} = \text{xor}_1(A_l) \rightarrow A_l \uparrow A_l \leftarrow \text{xor}_2(\text{xor}_1(A_l)). \quad (22)$$

While the sequence of directions produced by this method is the same as the one produced by the classical Lindenmayer-System, this method incurs an unnecessary overhead, since the first quadrant of successive iterations must be recomputed, as depicted in Figure 6(a).

Alternatively, it is possible to incrementally build the next expansion step of a curve produced by axiom H or A , solely based on the current expansion step produced by axiom A or H , respectively. This insight is also a direct result of Propositions 2 and 3, since

$$H_{l+1} = A_l \uparrow \text{xor}_1(A_l) \rightarrow \text{xor}_1(A_l) \downarrow \text{xor}_2(A_l), \quad (23)$$

and

$$A_{l+1} = H_l \rightarrow \text{xor}_1(H_l) \uparrow \text{xor}_1(H_l) \leftarrow \text{xor}_2(H_l). \quad (24)$$

Note that if our curve is built in this manner, then the first quadrant of iteration $l + 1$ is represented by the curve computed in iteration l , as depicted in Figure 6(b), thus avoiding the previous overhead. However, it is important to understand that a curve generated by a Lindenmayer-System using axiom H or A will always end its traversal at coordinate pair $(2^l - 1, 0)$ or $(0, 2^l - 1)$, respectively. The successive axiom interchange of our method results in a different last-visited pair of coordinates for each new curve iteration, due to the different terminal symbols present in the first expansion step of axioms H and A . These terminal symbols represent the directions used to connect the four quadrants at a given iteration. From this point onward, we will refer to this type of direction as connecting directions and are represented by variables s_k , where $k \in \{1, 2, 3\}$ represents

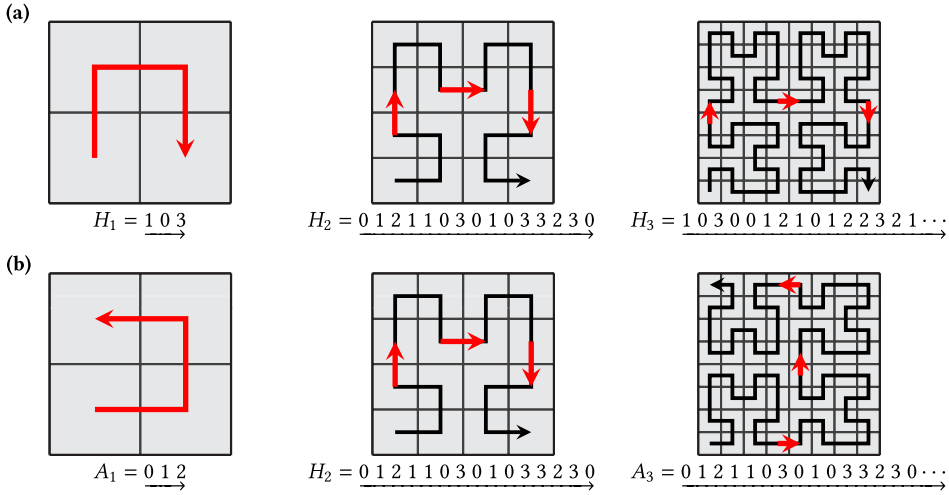


Fig. 6. Comparison between a recursive generation of the first three iterations of a Hilbert space-filling curve with axiom (a) H and a incremental generation of the same curve iterations by (b) axiom swapping.

the order of appearance of these terminals in the first expansion step. Proposition 4 formalizes the computation of connection directions s_1 , s_2 , and s_3 for incrementally generated curve.

PROPOSITION 4. *The values held by connecting direction variables s_1 , s_2 , and s_3 for iteration $l + 1$ can be calculated by applying a bit-wise exclusive-or by 1 to the connecting directions of iteration l .*

PROOF. Due to the incremental nature of our approach, whenever an odd iteration of a Hilbert space-filling curve is generated by axiom H , the last visited cell is $(2^l - 1, 0)$. In case we are generating an even iteration of this curve, for the same axiom, the last cell to be visited will be $(0, 2^l - 1)$ instead. The exact opposite happens for curves generated by axiom A . Since every curve starts its traversal on grid cell $(0, 0)$ and each quadrant must be completely traversed before passing onto the next one, a curve whose last visited cell is $(0, 2^l - 1)$ must visit first the lower-right quadrant, followed by the upper-right one, finishing its traversal on the upper-left quadrant. This corresponds to connecting directions $(s_1, s_2, s_3) = (\underline{0}_4, \underline{1}_4, \underline{2}_4)$. In case the last visited cell is $(2^l - 1, 0)$, our curve must first visit the upper-left quadrant, followed by the upper-right one, and finish its traversal on the lower-right quadrant. Implying the connecting directions are equal to $(s_1, s_2, s_3) = (\underline{1}_4, \underline{0}_4, \underline{3}_4)$.

Independently of the chosen axiom, at every generation iteration the connecting directions will alternate between $(\underline{0}_4, \underline{1}_4, \underline{2}_4)$ and $(\underline{1}_4, \underline{0}_4, \underline{3}_4)$. Concluding it is possible to compute the connecting directions for iteration $l + 1$ by performing a bit-wise exclusive-or by 1 to each connecting direction of iteration l ,

$$(s_1, s_2, s_3) := (\text{xor}_1(s_1), \text{xor}_1(s_2), \text{xor}_1(s_3)). \quad \square$$

In theory, Proposition 4 is not particularly relevant. The same curve could be computed through `if ... else` statements. It provides, however, an elegant solution to compute the s_k variables, reducing the number of branching instructions of this solution and contributing to the overall performance and readability of our solution.

3.3 Implementation Details

Propositions 2, 3, and 4 allow the construction of an iterative Hilbert space-filling curve generator in C, being able to be extended by SSE instructions due to the lack of data dependencies during

the construction step of each iteration. There are some crucial details regarding memory and the number of SSE instructions that were not yet addressed.

As previously stated, every direction symbol can be represented by 2 bits, but most C-like languages do not offer primitive types with such size. It is possible to avoid memory waste, and consequently reduce the number of SIMD instructions, by implementing a bit-array that stores the directions of a given Hilbert space-filling curve. Our bit-array is implemented as a dynamic array of `uint8_t`, a primitive type with 8 bits, meaning each entry of this array can store up to 4 directions. Given that iteration l of a Hilbert space-filling curve is composed by $4^l - 1$ directions, our bit-array requires exactly 4^{l-1} `uint8_t` variables to store a complete iteration. The following scheme illustrates how iteration l of a Hilbert space-filling curve is allocated in our bit-array `curve`, where variable d_k represents the k th direction to be processed:

$$\text{uint8_t } * \text{curve} := \underbrace{d_3 d_2 d_1 d_0}_{\text{curve}[0]}, \underbrace{d_7 d_6 d_5 d_4}_{\text{curve}[1]}, \dots, \underbrace{d_{4^l} d_{4^l-1} d_{4^l-2} d_{4^l-3}}_{\text{curve}[4^{l-1}]} \quad (25)$$

It might seem strange that the directions d_k , within each `uint8_t`, are allocated in little-endian style. This layout allows us to retrieve the directions contained by a `uint8_t` in $O(1)$ time by resorting to the remainder (%) and bit-wise shift-right (>>) operators. The retrieval of directions from our bit-array `curve` is depicted by the following nested-loop:

```

for  $h := 0; h < 4^{(l-1)}; h := h + 1$  do
    for  $k := 0; k < 4; k := k + 1$  do
        uint8_t  $t := \text{curve}[h];$ 
        int  $d := t \% 2;$ 
        int  $i := i - ((d - 2) \% 2);$ 
        int  $j := j - ((d - 1) \% 2);$ 
        /* Process entry (i,j) */
        ...
         $t := t >> 2;$ 

```

It is important to note that this layout modifies the location of connecting directions s_1 , s_2 , and s_3 in our bit-array. The connecting directions of a Hilbert curve with iteration l can be found after the first, second, and third quadrants of this curve, as seen in

$$H_l = A_{l-1} \underbrace{\uparrow}_{s_1} H_{l-1} \underbrace{\rightarrow}_{s_2} H_{l-1} \underbrace{\downarrow}_{s_3} B_{l-1}, \quad (26)$$

and

$$A_l = H_{l-1} \underbrace{\rightarrow}_{s_1} A_{l-1} \underbrace{\uparrow}_{s_2} A_{l-1} \underbrace{\leftarrow}_{s_3} C_{l-1}. \quad (27)$$

Implying variables s_1 , s_2 , and s_3 have to be allocated in $d_{4^{l-1}}$, $d_{2 \cdot 4^{l-1}}$, and $d_{3 \cdot 4^{l-1}}$ of our bit-array, which can be found in the two most-significant bits of the entries `curve[ $4^{l-2} - 1$ ]`, `curve[ $2 * (4^{l-2}) - 1$ ]`, and `curve[ $3 * (4^{l-2}) - 1$ ]`, respectively. If we unset these bits, then it is possible to insert the connection directions in our bit-array through a bit-wise or operation (`|`):

$$\text{curve}[4^{l-2} - 1] := \text{curve}[4^{l-2} - 1] | s_1; \quad (28)$$

$$\text{curve}[(2 * 4^{l-2}) - 1] := \text{curve}[(2 * 4^{l-2}) - 1] | s_2; \quad (29)$$

$$\text{curve}[(3 * 4^{l-2}) - 1] := \text{curve}[(3 * 4^{l-2}) - 1] | s_3; \quad (30)$$

Since each connecting direction is now stored in a `uint8_t` variable, the computation of these directions for the next curve iteration slightly differs from that of Proposition 4. In this situation,

we are only required to apply a bit-wise exclusive-or ($\wedge$) by 1 to the second most significant bit of our connecting directions. This operation can be achieved by resorting to the binary bit-mask $0b01000000$:

$$s_1 := s_1 \wedge 0b01000000; \quad (31)$$

$$s_2 := s_2 \wedge 0b01000000; \quad (32)$$

$$s_3 := s_3 \wedge 0b01000000; \quad (33)$$

Algorithm 1 illustrates the implementation of our Hilbert space-filling curve generator in C extended with SSE Intrinsics [14]. To keep this pseudo-code readable, we opted to represent each `uint8_t` literal in base-4, explicitly depicting the directions contained by this variable, e.g., the sequence of directions $\underline{0\ 1\ 2\ 3}_4 = \underline{3\ 2\ 1\ 0}_4 = 228_{10}$. We will now explain the most important implementation details of our solution:

- **Memory Allocation** (Line 1): Our generator starts by allocating memory for our bit-array curve based on the `level` input parameter that represents the desired level of expansion of the curve. As previously observed, our bit-array requires $4^{\text{level}-1}$ `uint8_t` variables to store the directions of a Hilbert curve with level equal to `level`.
- **Bit-array Initialization** (Lines 2 to 11): Depending on the axiom that was passed as input, our generator initializes the bit-array curve with the directions that characterize the third level of expansion of a Hilbert space-filling curve produced by axiom. This curve iteration is allocated following the scheme presented in Equation (25). Note that a curve with this level of expansion completely fills an SSE vector (128 bits), which allows us to avoid corner cases associated with smaller iterations of the Hilbert curve.
- **Connecting Directions Initialization** (Lines 4 to 11): At this stage, the connecting directions s_1 , s_2 , and s_3 are also initialized depending on the axiom value. These variables are represented as `uint8_t` variables, as previously stated.
- **Exclusive-or Mask** (Lines 12 and 13): Initializes two SSE integer vectors (`__m128i`) containing the bit-masks required to apply a bit-wise exclusive-or by 1 (`mask_xor1`), or 2 (`mask_xor2`), to all directions contained by another SSE integer vector. `mask_xor1` corresponds to an SSE vector with eight `uint8_t` all equal to $\underline{1\ 1\ 1\ 1}_4$, while `mask_xor2` corresponds to another SSE vector composed by eight `uint8_t` with values equal to $\underline{2\ 2\ 2\ 2}_4$.
- **Generating Next Curve Iteration** (Lines 14 to 31):
 - **Identification of Quadrant Bounds** (Lines 15 to 22): Variables `offset_q2`, `offset_q3` and `offset_q4` store the indexes of the first `uint8_t` entries of our bit-array curve that contain data regarding quadrants 2, 3, and 4, respectively.
 - **Computing the Last Three Quadrants** (Lines 18 to 22): As seen in Propositions 2 and 3, it is possible to generate the last three quadrants of iteration l by the application of `xor1` and `xor2` to the first quadrant of this iteration. Since this curve is built incrementally, the first quadrant of iteration l is the same as the curve computed in iteration $l - 1$. These functions can be encoded by the SSE Intrinsic `_mm_xor_si128`, which takes two `__m128i` vectors as arguments and performs a bit-wise exclusive-or to the elements of these vectors. In our application, the input vectors are q_1 , that corresponds to a chunk of the data contained by the first quadrant of this iterations, and `mask_xor1` or `mask_xor2`, depending whether we want to apply an exclusive-or by 1 or by 2 to the directions within q_1 . Once this bit-wise operation is concluded, the resulting vector is then stored in the appropriate curve entry by resorting to the previously computed `offset_q2`, `offset_q3` and

ALGORITHM 1: SIMD Hilbert Space-filling Curve Generator**Input:** axiom, level**Output:** curve**begin**

```

1  uint8_t *curve := _mm_malloc(4level-1 * sizeof(uint8_t), ...);
2  if axiom = H then
3      curve[0] := 0 3 0 14; ... ; curve[15] := 0 3 0 14;          /* Initialize curve := H3 */
4      uint8_t s1 := 1 0 0 04;
5      uint8_t s2 := 0 0 0 04;
6      uint8_t s3 := 3 0 0 04;
7  else
8      curve[0] := 1 2 1 04; ... ; curve[15] := 0 2 1 04;          /* Initialize curve := A3 */
9      uint8_t s1 := 0 0 0 04;
10     uint8_t s2 := 1 0 0 04;
11     uint8_t s3 := 2 0 0 04;
12     __m128i mask_xor1 = _mm_set_epi8(1 1 1 14, ..., 1 1 1 14);
13     __m128i mask_xor2 = _mm_set_epi8(2 2 2 24, ..., 2 2 2 24);
14     for unsigned int l := 3; l < level; l := l + 1 do
15         unsigned int offset_q2 := 4l-1;
16         unsigned int offset_q3 := 2 * 4l-1;
17         unsigned int offset_q4 := 3 * 4l-1;
18         for unsigned int c := 0; c < 4l-1; c := c + 16 do
19             __m128i q1 := _mm_loadu_si128(&curve[c]);
20             _mm_storeu_si128(&curve[offset_q2 + c], _mm_xor_si128(q1, mask_xor1));
21             _mm_storeu_si128(&curve[offset_q3 + c], _mm_xor_si128(q1, mask_xor1));
22             _mm_storeu_si128(&curve[offset_q4 + c], _mm_xor_si128(q1, mask_xor2));
23         curve[offset_q2 - 1] := curve[offset_q2 - 1] & 0 1 1 14;
24         curve[offset_q3 - 1] := curve[offset_q3 - 1] & 0 1 1 14;
25         curve[offset_q4 - 1] := curve[offset_q4 - 1] & 0 1 1 14;
26         curve[offset_q2 - 1] := curve[offset_q2 - 1] | s1;
27         curve[offset_q3 - 1] := curve[offset_q3 - 1] | s2;
28         curve[offset_q4 - 1] := curve[offset_q4 - 1] | s3;
29         s1 := s1 ^ 1 0 0 04;
30         s2 := s2 ^ 1 0 0 04;
31         s3 := s3 ^ 1 0 0 04;

```

offset_q4. Finally, it is important to note that this step is repeated until all curve entries related to the first quadrant are processed.

- **Resetting the Connecting Directions Bits** (Lines 23 to 25): Due to the application of functions xor₁ and xor₂, the bits of our bit-array that correspond to connecting directions

s_1 , s_2 , and s_3 will not be clear. To make sure that these directions are properly allocated in our bit-array, we must first reset the corresponding bits. This procedure is implemented as a bit-wise and (&) between the entries that contain a connecting direction and bit-mask $0b00111111$ ($\underline{0\ 1\ 1\ 1_4}$).

- **Allocating Connecting Directions** (Lines 26 to 28): Once the bits where the connecting direction will be allocated are reset, it is possible to insert these directions through a bit-wise or (|) as shown in Equations (29) to (30).
- **Setting Up the Next Iteration** (Lines 29 to 31): At this point, the first $4^{l+1} - 1$ pairs of bits of our bit-array curve contain the directions that characterize iteration $l + 1$ of a Hilbert space-filling curve. Computing further iterations of this curve is possible by repeating this process from Line 16. However, due to the incremental nature of our generator, we are required to update the value of the connecting directions s_1 , s_2 , and s_3 following Proposition 25 and Equations (31) to (33).

3.4 Time and Memory Analysis

PROPOSITION 5. *Algorithm 1 presents a time complexity of $O(N)$, where N is the number of grid entries.*

PROOF. Let $l \in \mathbb{N}^+$ represent the level of recursion, or iteration number, of a given Hilbert space-filling curve, and $a, b \in \mathbb{N}^+$ the number of constant time operations within the outer loop (Line 14) and the inner-most loop (Line 18) of Algorithm 1, respectively. The total number of operations to compute a curve of level l corresponds to the following sum:

$$S_l = b(l - 4) + \sum_{i=3}^{l-1} a(4^i - 1).$$

Sum S'_l is a geometric progression and an upper-bound of S_l

$$S'_l = b(l - 4) + \sum_{i=3}^{l-1} a(4^i) > b(l - 4) + \sum_{i=3}^{l-1} a(4^i - 1) = S_l.$$

Since $l = \log_4(N)$, this geometric progression can be further simplified as

$$S'_l = b(\log_4(N) - 4) + \frac{a(N - 1)}{3}.$$

Since $N > \log_4(N)$, we conclude that $S'_l \in O(N)$. Which implies that $S_l \in O(N)$, given that S'_l is an upper-bound of S_l . $\square$

PROPOSITION 6. *Algorithm 1 presents a memory complexity of $O(N)$, where N is the number of grid entries.*

PROOF. Algorithm 1 is iterative and only allocates dynamic memory during the initialization step of our bit-array in Line 1. This bit-array contains exactly $2N$ bits, which results in a memory cost of $O(N)$. $\square$

4 BLOCKING SCHEME FOR HILBERT AND Z-ORDER SPACE-FILLING CURVES

To generalize the Hilbert curve to rectangular grids, Böhm et al. [5] implemented a solution that uses small pre-processed curves called nano-programs. Each nano-program represents a sequence of directions that forms a curve able to fill grids with side length ranging from 1 to 4. This path is then completed by tessellation, applying the correct nano-program between any two sequential

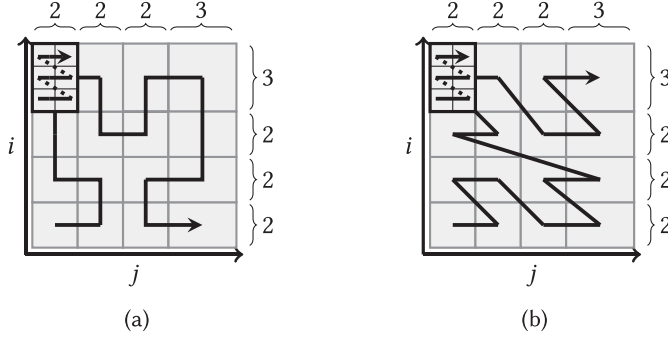


Fig. 7. Blocking-scheme example of two 9×9 grids using a traversal based on the second iteration of (a) Hilbert and (b) Z-order space-filling curves. The entries of each block are visited following a row-major order traversal.

directions of a Hilbert curve. The use of nano-programs is an ingenious way to fill rectangular grids while preserving locality inside its sub-grids. However, we suspect it is possible to reduce the nano-programs to a row-major order scan without compromising locality, as depicted by Figure 7, if the sub-grids are small enough and the host algorithm presents no spatial locality, which is the case for the out-of-place matrix transposition problem. Proposition 7 ensures that it is possible to block any $n \times m$ grid, for $\lfloor \log_2(n) \rfloor = \lfloor \log_2(m) \rfloor$, $n > 1$ and $m > 1$, into a square $2^{\lfloor \log_2(m) \rfloor - 1} \times 2^{\lfloor \log_2(m) \rfloor - 1}$ partitioned grid. The dimension of this grid allows to traverse its blocks following the order imposed by a Hilbert or Z-order curve.

PROPOSITION 7. *It is possible to split a $n \times m$ grid, where $n > 1$, $m > 1$ and $\lfloor \log_2(n) \rfloor = \lfloor \log_2(m) \rfloor$, into $2^{\lfloor \log_2(n) \rfloor - 1} \times 2^{\lfloor \log_2(m) \rfloor - 1}$ blocks whose sides range from 2 to 4.*

PROOF. This proposition statement can be proven through the following system of equations, where **length** is the size of the original grid side n or m . Variables $a, b \in \mathbb{N}_0$ represent the number of segments of size 2 and 3 or 3 and 4,

$$2^{\lfloor \log_2(\text{length}) \rfloor - 1} = a + b \quad (34)$$

and

$$\begin{cases} \text{length} = 2a + 3b & \text{if } \text{length} \leq \frac{3}{2} 2^{\lfloor \log_2(\text{length}) \rfloor}, \\ \text{length} = 3a + 4b & \text{otherwise.} \end{cases} \quad (35)$$

Equation (34) ensures the number of segments in a side of size **length** is equal to $2^{\lfloor \log_2(\text{length}) \rfloor - 1}$, which is a power of 2. While Equation (35) ensures the sum of the size of all segments is equal to the original side length. Applying several substitution steps to this system results in

$$\begin{cases} a = -\text{length} + \frac{3}{2} 2^{\lfloor \log_2(\text{length}) \rfloor} \\ b = \text{length} - 2^{\lfloor \log_2(\text{length}) \rfloor} \end{cases} \quad \text{if } \text{length} \leq \frac{3}{2} 2^{\lfloor \log_2(\text{length}) \rfloor}, \quad (36)$$

$$\begin{cases} a = -\text{length} + 2^{\lfloor \log_2(\text{length}) \rfloor + 1} \\ b = \text{length} - \frac{3}{2} 2^{\lfloor \log_2(\text{length}) \rfloor} \end{cases} \quad \text{otherwise.} \quad (37)$$

Variables a and b , in Equations (36) and (37), are well-defined for **length** > 1 , and $2^{\lfloor \log_2(\text{length}) \rfloor}$ is divisible by 2, ensuring this system always has an integer solution for a and b . Concluding side n and m , of the original grid, can be divided into $2^{\lfloor \log_2(n) \rfloor - 1}$ and $2^{\lfloor \log_2(m) \rfloor - 1}$ segments whose size ranges between 2 and 4. $\square$

Algorithm 2 depicts our strategy to generalize square space-filling curves whose sizes are a power of 2, such as the Hilbert and Z-order space-filling curves, to $n \times m$ grids where $\lfloor \log_2(n) \rfloor = \lfloor \log_2(m) \rfloor$. The main benefit of this blocking strategy is that it allows the employment of standard prefetching techniques in a timely manner, which greatly improves the performance of the out-of-place transposition problem. The most important details of our solution are:

- **Side Splicing** (Lines 2 to 3): The first step of our blocking procedure identifies whether the grid sides will be spliced into segments of length 2 and 3 or 3 and 4. If a grid side respects $\text{length} \leq \frac{3}{2}2^{\lfloor \log_2(\text{length}) \rfloor}$, then this side will be spliced into segments of length 2 and 3. In case this inequality does not hold, this side will then be spliced into segments of length 3 and 4. The type of partition used in each side is encoded by variables n_type and m_type . Once the partition type is identified, it is possible to obtain the number of smaller segments (n_0 and m_0) and larger segments (n_1 and m_1) by solving Equation System (36) or (37) of Proposition 7, depending on the type of partition of that dimension.
- **Block Coordinates** (Line 11): The coordinate pair (I, J) identifies a single block within our partitioned grid. This pair of coordinates corresponds to the coordinates of the h th step of a given space-filling curve, which is computed by `get_block_coordinates`. This function can be computed by resorting to our SIMD Hilbert Space-filling Generator, or to the non-recursive Lindenmayer-System proposed by Böhm et al. [5], for a traversal based on the Hilbert curve. To create a traversal based on the Z-order curve, we implemented the kernel provided in libmorton [4] that leverages BMI2 `_pext_u32` instructions to efficiently compute (I, J) in constant time and memory:

$I := _pext_u32(h, 0xAAAAAAAA);$

$J := _pext_u32(h, 0x55555555);$

- **Block Boundaries** (Lines 12 to 13): Variables i_min , i_max , j_min , and j_max represent the bounds of a given grid block. The values of these bounds are calculated by

ALGORITHM 2: $2^l \times 2^l$ Space-filling Curve Blocking-Scheme

Input: m and n

begin

```

1  level :=  $\lfloor \log_2(\min(n, m)) \rfloor - 1$ ;
2  ( $n\_0, n\_1, n\_type$ ) := split_side( $n$ );
3  ( $m\_0, m\_1, m\_type$ ) := split_side( $m$ );
4  ( $I, J$ ) := (0, 0);
5  ( $i\_min, j\_min$ ) := (0, 0);
6  if  $n\_0 > 0$  then  $i\_max := n\_0$ ; else  $i\_max := n\_1$ ;
7  if  $m\_0 > 0$  then  $j\_max := m\_0$ ; else  $j\_max := m\_1$ ;
8  for  $h := 0$ ;  $h < 4^{\text{level}} - 1$ ;  $h := h + 1$  do
9      ( $i\_min\_prev, i\_max\_prev$ ) := ( $i\_min, i\_max$ );
10     ( $j\_min\_prev, j\_max\_prev$ ) := ( $j\_min, j\_max$ );
11     ( $I, J$ ) := get_block_coordinates( $h$ );
12     ( $i\_min, i\_max$ ) := compute_boundaries( $I, n\_0, n\_1, n\_type$ );
13     ( $j\_min, j\_max$ ) := compute_boundaries( $J, m\_0, m\_1, m\_type$ );
14     prefetch_next_block( $i\_min, i\_max, j\_min, j\_max$ );
15     process_block( $i\_min\_prev, i\_max\_prev, j\_min\_prev, j\_max\_prev$ );

```

`compute_boundaries` that returns the boundaries of this block, based on (I, J) and the shape of our partitioned grid. The boundaries of a given block can be obtained in constant time and memory by resorting to the following kernel:

```
j_max := (m_0 * min(J + 1, m_0)) + (m_1 * max(0, J + 1 - m_0));
i_max := (n_0 * min(I + 1, n_0)) + (n_1 * max(0, I + 1 - n_0));
j_min := (m_0 * min(J, m_0)) + (m_1 * max(0, J - m_0));
i_min := (n_0 * min(I, n_0)) + (n_1 * max(0, I - n_0));
```

- **Prefetching** (Line 14): Before processing the entries of the current block, our solution identifies the boundaries of the next block to be processed and prefetches the cache lines that correspond to the first entry of each block row. This step is encoded by `prefetch_next_block`, which employs GCC `__builtin_prefetch(addr, rw, locality)` [21]. Argument `addr` represents the memory address we want to bring to memory. `rw` is a compile-time constant that indicates whether the prefetch data will be used for a read (0) or write (1) operation. `locality` is also a compile-time constant that ranges from 0 to 3, where 0 states that the prefetch data has no temporal locality, this cache-line will be evicted once it is accessed, while 3 states that the prefetch data has the highest degree of temporal locality and will be persistent in all cache levels.
- **Block Processing** (Line 15): Once the prefetching step is concluded our solution starts to process the entries of the current block, based on the previously assigned variables `i_min_prev`, `i_max_prev`, `j_min_prev`, and `j_max_prev`. This block is then processed in row-major order.

4.1 Severely Asymmetrical Grids

Proposition 7 is not verified for severely asymmetrical grids, i.e., $\lfloor \log_2(n) \rfloor \neq \lfloor \log_2(m) \rfloor$. The strategy used to traverse grids with these dimensions is similar to the one employed by Böhm et al. [5]. Our objective is to splice the original $n \times m$ grid into several $n' \times m'$ sub-grids, such that $\lfloor \log_2(n') \rfloor = \lfloor \log_2(m') \rfloor$. Proposition 7 holds for each $n' \times m'$ sub-grids, implying it is possible to traverse all of them with iteration $\min(n, m) - 1$ of a Hilbert or Z-order curve. Proposition 8 ensures such split is possible for grids where $\lfloor \log_2(n) \rfloor \neq \lfloor \log_2(m) \rfloor$, $n > 1$ and $m > 1$.

PROPOSITION 8. *Any $n \times m$ grid, where $\lfloor \log_2(n) \rfloor \neq \lfloor \log_2(m) \rfloor$, can be split into $\left\lfloor \frac{\max(n, m)}{2^{\lfloor \log_2(\min(n, m))}} \right\rfloor$ sub-grids, whose dimensions respect $\lfloor \log_2(\min(n, m)) \rfloor = \lfloor \log_2(\text{height}) \rfloor = \lfloor \log_2(\text{width}) \rfloor$.*

PROOF. Assume an initial $n \times m$ grid, where $n > 1$, $m > 1$ and $\log_2(n) < \log_2(m)$. It is then possible to rewrite m as

$$m = \left(\left(\left\lfloor \frac{m}{2^{\lfloor \log_2(n) \rfloor}} \right\rfloor - 1 \right) 2^{\lfloor \log_2(n) \rfloor} \right) + \left(2^{\lfloor \log_2(n) \rfloor} + m \bmod 2^{\lfloor \log_2(n) \rfloor} \right). \quad (38)$$

Equation (38) suggests it is possible to split this horizontally severely asymmetrical grid into one sub-grid with dimensions $n \times (2^{\lfloor \log_2(n) \rfloor} + m \bmod 2^{\lfloor \log_2(n) \rfloor})$ sub-grid, plus $\lfloor m/2^{\lfloor \log_2(n) \rfloor} \rfloor - 1$ sub-grids with dimensions $n \times 2^{\lfloor \log_2(n) \rfloor}$. Since $\lfloor \log_2(n) \rfloor = \lfloor \log_2(2^{\lfloor \log_2(n) \rfloor}) \rfloor = \lfloor \log_2(2^{\lfloor \log_2(n) \rfloor} + m \bmod 2^{\lfloor \log_2(n) \rfloor}) \rfloor$, Proposition 7 holds for all sub-grids. $\square$

Proposition 8 implies it is possible to extend our blocking procedure to severely asymmetrical grids by employing Algorithm 2 to each $n' \times m'$ grid, while updating the values of variables `i_min` and `j_min` accordingly.

4.2 Inter-grid Locality

As observed in Section 3.2, the last cell visited by a Hilbert space-filling curve generated by axiom H will always be $(2^l - 1, 0)$, while the last cell visited by a curve generated by axiom A is $(0, 2^l - 1)$. This fact allows us to leverage the curve creation procedure to improve inter-grid locality in case we want to process a severely asymmetrical grid. To preserve inter-grid locality, our strategy must ensure that the two last quadrants of a given sub-grid are adjacent to the first two quadrants of the next sub-grid. The first step to satisfy this restriction is to identify whether our grid is horizontally, $\log_2(n) < \log_2(m)$, or vertically, $\log_2(n) > \log_2(m)$, severely asymmetrical. If our grid is horizontally asymmetrical, then the last visited block of each sub-grid will be $(k(2^l - 1), 0)$, where $k \in \mathbb{N}$ represents current sub-grid. In case our grid is vertically severely asymmetrical, $(0, k(2^l - 1))$ will be the last visited sub-grid block. Following the observations of Proposition 4, if our curve is horizontally severely asymmetrical, then our curve must be generated by axiom H or A for odd or even values of l , respectively. However, if our curve is vertically asymmetrical, then it must be generated by axiom A or H for odd or even values of l . Finally, it is important to note that a traversal based on the Z-order curve always ends in a cell that is diagonally opposite to its starting point, meaning it is impossible to leverage the generation of this curve to improve inter-grid locality.

4.3 Time and Memory Analysis

PROPOSITION 9. *Our blocking scheme, Algorithm 2, presents $O(nm)$ time complexity, where n and m correspond to the number of rows and columns of its input grid.*

PROOF. The original $n \times m$ grid is partitioned into non-overlapping blocks by function `split_side`, which costs $O(1)$ time and is called once per each Hilbert space-filling curve generated. The boundaries of each block are computed by `get_block_coordinates`, which also costs $O(1)$ time. Finally, the function `prefetch_new_block` issues a number of software prefetching instructions proportional to the number of rows of a given block. Consequently, the complexity of Algorithm 2 is proportional to the input grid size $O(nm)$. $\square$

PROPOSITION 10. *Our blocking scheme, Algorithm 2, presents $O(nm)$ memory complexity for curves generated by our SIMD Hilbert Space-filling Curve Generator, non-recursive Lindenmayer-System [5], or the `_pext_u32` function.*

PROOF. Independently of the generator chosen, Algorithm 2 assumes the existence of a $n \times m$ grid in memory. If the curve used by our blocking scheme was generated by the non-recursive Lindenmayer-System or by `_pext_u32`, then no further memory is dynamically allocated. Therefore, the memory complexity of Algorithm 2 is $O(nm)$. If the curve used by this algorithm was generated by our SIMD Hilbert Space-filling Curve Generator, then Algorithm 2 incurs on overhead of 2 bits per block. Since the number of blocks of any grid are smaller than the total number of entries, the memory complexity of this algorithm remains equal to $O(nm)$. $\square$

5 EXPERIMENTAL EVALUATION

The following experiments were performed in an Intel Xeon Gold 6130 (Skylake) CPU, with 2.1 GHz fixed clock frequency, 32 KiB (8-ways) private L1 data cache, 1 MiB (16-ways) private L2 cache, and 22 MiB (11-ways) shared L3 cache. All cache levels employ a Write-Back policy. This machine contains 6 memory channels and presents a maximum memory speed of 2.6 GHz. The operating System installed in this machine is the CentOS Linux 7, version 3.10.0-1127.19.1.el7.x86_64. All experiments were written in C and compiled with GNU GCC Red Hat 9.4.0. Memory metrics were evaluated with `perf` version 3.10.0.

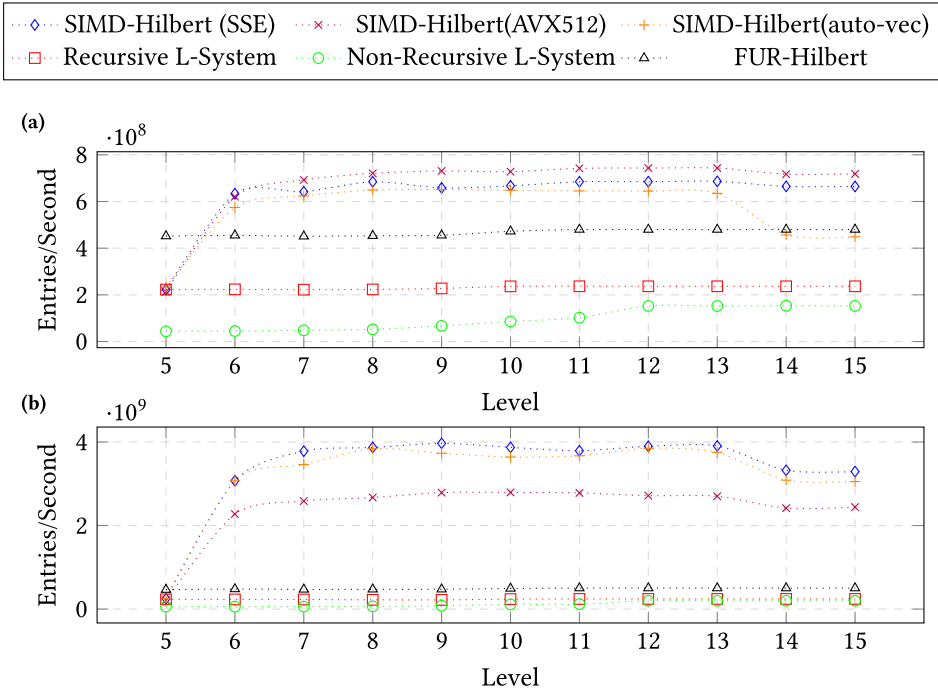


Fig. 8. Execution time comparison of different Hilbert space-filling curve enumerators, from iteration 5 to 15, with optimization flags (a) -O1 -march=native and (b) -O3 -march=native.

5.1 Hilbert Curve Generators

Three versions of our SIMD Hilbert Space-filling Curve Generator, Algorithm 1, were implemented to assess the performance difference between employing SSE or AVX512 instructions and a compiler auto-vectorized implementation. These solutions were compared against known methods that enumerate all points of a given Hilbert space-filling curve in linear time, such as the non-recursive and recursive Lindenmayer-System and FUR-Hilbert.

Figures 8(a) and 8(b) depict the average of a total of 15 runs of the number of Hilbert space-filling curve entries processed per second for each approach, compiled with -O1 -march=native and -O3 -march=native, respectively. It is important to note that a complete Hilbert curve is computed per run. This analysis does not exploit the possible amortization of Algorithm 1. Furthermore, each test iterates over all (i, j) coordinate-pairs that correspond to a point of a given iteration of a Hilbert space-filling curve.

5.1.1 Compiler Optimization -O1. The best-performing approach for optimization level -O1 is the AVX512 implementation of SIMD-Hilbert, which presents a global average of $6.70 \cdot 10^8$ entries per second, and an average of $7.32 \cdot 10^8$ entries per second from level 10 onward. The implementation of this solution by resorting to SSE instructions comes as a close second, processing on average 6.47% less entries per second than its AVX512 counterpart. The auto-vectorized version of our solution is 9.91% slower than the SSE version for the last five levels. All SIMD-Hilbert implementations outperform FUR-Hilbert from level 6 onward. Our AVX512 and SSE implementations are on average 1.53 $\times$ and 1.41 $\times$ faster than FUR-Hilbert, for the last five levels.

5.1.2 Compiler Optimization -O3. The optimization flag -O3 significantly reduced the runtime of the different SIMD Hilbert Space-filling Generator versions. The approaches that benefited the

most from compiler optimizations were the auto-vectorized implementation of our approach, with a $5.70\times$ speedup, and its SSE counterpart, which presents a similar speedup of $5.37\times$. The compiler optimized AVX512 version of our generator presented lower performance gains with a speedup of $3.58\times$. The compiler was not able to optimize the remaining approaches as efficiently. FUR-Hilbert presents a speedup of $1.04\times$, while the recursive and non-recursive versions of the Lindenmayer-System present a speedup of $1.02\times$ and $1.27\times$. The most performant approaches for this optimization level are the SSE and auto-vectorized implementations of our SIMD Hilbert Space-filling Curve Generator. From level 10 onward, the SSE version computes an average of $3.68 \cdot 10^9$ curve points per second, while the auto-vectorized version presents an average of $3.51 \cdot 10^9$ points per second. On average, the SSE implementation computes 4.55% more curve points per second than our auto-vectorized version. The AVX512 version of our generator is the third most performant approach, achieving 71.26% and 74.51% of the throughput presented by SSE and auto-vectorized implementations, respectively. The different implementations of Algorithm 1 when optimized by the compiler present great performance improvements when compared to FUR-Hilbert and the two Lindenmayer-System versions. Most notably, the SSE implementation of our SIMD Hilbert Space-filling Curve Generator, which is $6.90\times$ faster than FUR-Hilbert, $27.63\times$ and $14.32\times$ faster than the non-recursive and recursive Lindenmayer-Systems.

5.2 Out-of-place Transposition

This section assesses the performance of five different approaches that compute the out-of-place transpose of rectangular single-precision matrices. The evaluated approaches consist of a naive transposition strategy [19]; a cache-oblivious adaptation of FUR-Hilbert [5]; `somat_copy`, a cache-aware hand-tuned transposition strategy offered by Intel MKL [18]; and two versions of our cache-oblivious blocking scheme, which employs a traversal based on a Hilbert or Z-order curve. Unless it is explicitly stated, our Hilbert blocking scheme corresponds to a version of our blocking scheme, whose curve was generated by the SSE version of our SIMD Hilbert Space-filling Curve Generator. To further optimize these implementations, compilation flags `-O3` and `-march=native` were applied.

5.2.1 SIMD Parallelism. It is important to note that the solution offered by Intel MKL exploits SIMD parallelism. To obtain competitive results, the remaining approaches were extended with an SSE intrinsic transposition kernel [14] that performs the transposition of a 4×4 sub-matrix. The utilization of this kernel implies that every visited grid entry corresponds to a sub-matrix of this size. As is, these implementations are only able to compute the transpose of a rectangular matrix whose sizes are a multiple of 4. Memory padding is required to deal with matrices of different sizes.

5.2.2 Software Prefetching. The implementation of software prefetching by our blocking scheme was performed by GCC building `__builtin_prefetch`, with locality hint equal to 3. We did not notice any significant difference between the execution time of the different locality values. Even though the implementation of Intel MKL is closed-source, we expect this library to leverage software prefetching techniques.

5.2.3 Time Benchmarks. Execution time measurements were performed by resorting to the wall-clock timer provided by the `time.h` library. This measurement does not consider the time required to initialize the input and output matrices and is the result of the average of five independent runs. The performance of each approach is expressed as throughput (GiB/s), calculated as

$$\text{throughput} = \frac{3 \cdot \text{matrix_elements} \cdot \text{sizeof(float)}}{2^{30} \cdot \text{time}}.$$

STREAM [17] computes the throughput of its benchmarks by counting the amount of memory that was loaded and stored during the execution. In the context of the out-of-place transposition we incur on 2 reads and 1 store per entry transposed, thus the constant multiplier in our formula. This formula can be directly compared to the results obtained by the STREAM benchmark suite, more specifically the COPY benchmark. COPY provides the practical peak memory throughput of a given system for the out-of-place copy procedure of a large array. This benchmark does not issue arithmetic instructions, providing an upper-bound of the maximum throughput achievable by a matrix transposition procedure. The COPY benchmark returns a throughput of 10.6 GiB/s on this experimental setup. The efficiency of a given approach on this system is calculated as

$$\frac{\text{approach_throughput}}{\text{COPY_throughput}}.$$

5.2.4 Memory Benchmarks. It is possible to characterize the global memory hierarchy access behavior by evaluating PMU counters `cycle_activity.stalls_mem_any:u`, which counts the number of execution stalls while a memory load instruction is pending, and `resource_stalls.sb:u`, which counts the number of stalled cycles while the store buffer was full. The suffix `:u` signifies these events were measured in user-space. These events might occur simultaneously, implying that a single stall might be counted by both events. Nonetheless, the sum of these counters should result in a good estimate of memory related stalls. These values are used to calculate the reduction of memory-related stalls of the cache-friendly approaches when compared to naive transposition method. This value is calculated as

$$\frac{\text{naive_stalls}}{\text{approach_stalls}}.$$

5.2.5 Matrix Sizes. We borrowed the notation used by Vladimirov [23] to divide the performance analysis into good and ugly matrices. Good matrices are expected to incur on a low count of conflict misses, meaning most memory stalls will be the result of cold and capacity misses. Ugly matrices incur on a large number of conflict misses, mapping the memory addresses corresponding to matrix data to a reduced number of cache sets, which renders a large fraction of the cache unused. Ugly matrices correspond to matrices whose sides are a multiple of the critical stride. This stride, as explained by Fog [10], can be calculated as

$$\text{critical_stride} = \frac{\text{cache_size}}{N\text{-ways} \cdot \text{sizeof}(\text{TYPE})}.$$

The size of a single-precision float on this Intel Xeon Gold 6130 is 4 Bytes, which results in a critical stride of 1,024 elements for cache L1d, 16,384 elements for cache L2, and 524,288 elements for the last level of cache. Naturally, the amount of conflict misses further increases for sizes that correspond to multiples of two or more cache critical strides. The matrices selected for the following experiments are composed by single-precision floating point numbers and present side lengths ranging between 8,192 and 16,384. Experiments with tall and wide non-square matrices are averaged together to keep the evaluation succinct. The largest dimension for non-rectangular matrices remains constant for all tests. The largest dimension contains 16,100 and 16,834 elements for good and ugly matrices. Note that none of the matrix sizes tested fit in cache.

5.2.6 Evaluation: Good Square Matrices. Figures 9(a) and 9(b) depict the throughput and the reduction of memory related stalls of the tested approaches for good-sized square matrices. As expected, the cache-efficient approaches provided significant performance gains over the naive cache-ignorant solution. Surprisingly, both FUR-Hilbert and our Hilbert curve blocking scheme presented significant gains over the Intel MKL cache-aware solution. Our Hilbert curve blocking scheme, extended with standard prefetching techniques, presented the best performance with an

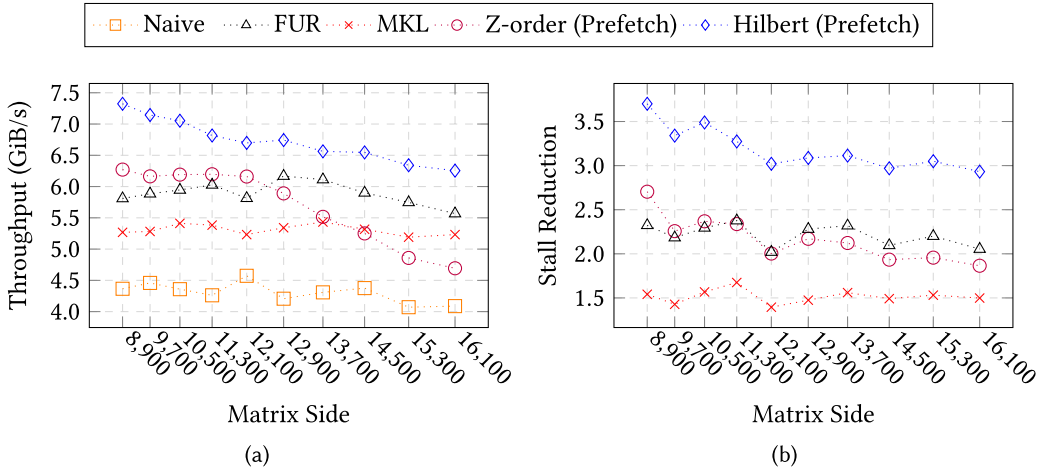


Fig. 9. (a) throughput in GiB/s and (b) memory-related stall reduction, in comparison to the naive approach, of the different out-of-place matrix transposition strategies for good square matrix sizes.

average throughput of 6.75 GiB/s (an efficiency of 64%) and a $3.17\times$ memory-related stalls reduction. The application of standard prefetching techniques to this approach resulted in an average speedup of 31%.

Our Z-order blocking scheme, also extended with the same prefetching techniques, presented the second-best performance for matrices with dimensions smaller than $12,900 \times 12,900$, presenting an average throughput of 6.15 GiB/s. The performance of this approach steadily decayed for larger matrices, being outperformed by FUR-Hilbert for matrices with dimensions larger than $12,100 \times 12,100$, and by MKL for matrices with dimensions larger than $14,500 \times 14,500$.

The application of software prefetching to this approach resulted in an average speedup of 24%. Overall, the second best-performing approach was FUR-Hilbert, which presents a steady throughput average of 5.85 GiB/s. Finally, it is worth to mention that the execution time and stall reduction of Intel MKL seem to not be correlated. The closed-source nature of Intel MKL implies it impossible to understand how this method was implemented and subsequently which PMU counters should be analyzed.

5.2.7 Evaluation: Ugly Square Matrices. Figures 10(a) and 10(b) depict the throughput and memory-related stalls reduction of the tested approaches for ugly-sized square matrices. Every matrix size present in this analysis is a multiple of the critical stride of cache L1. It is important to note that matrices with side length equal to 16,384 are also multiples of cache L2 critical stride, further increasing the number of conflict misses. While it is possible to observe a decay in performance by every approach, when compared to the results presented in Section 5.2.6, the solution provided by Intel MKL was the only that presented a severe performance drop for matrices whose strides are a multiple of both L1 and L2 critical strides. The best-performing approach for ugly matrices was our Hilbert curve blocking scheme, again extended with standard prefetching techniques, presenting an average throughput of 6.15 GiB/s (an efficiency of 58%) and a stall reduction of $1.75\times$, for matrices whose dimensions range between $9,216 \times 9,216$ and $15,360 \times 15,360$. This approach is closely followed by our Z-order blocking scheme, which presents an average throughput of 5.96 GiB/s and a stall reduction of $1.78\times$. The application of software prefetching for ugly-sized squared matrices, resulted in a speedup of 28% and 26% when compared to the non-prefetching versions of the Hilbert and Z-order blocking schemes, respectively. We thought FUR-Hilbert would

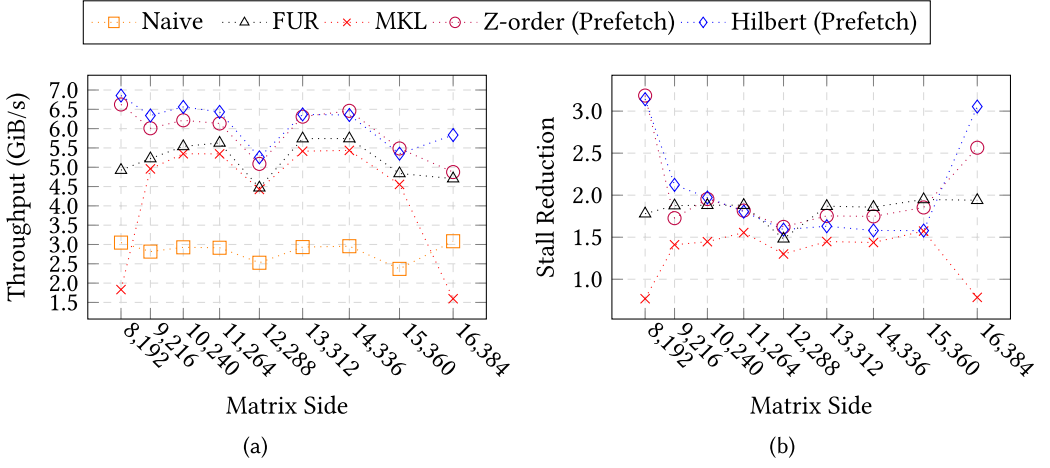


Fig. 10. (a) throughput in GiB/s and (b) memory-related stall reduction, in comparison to the naive approach, of the different out-of-place matrix transposition strategies for ugly square matrix sizes.

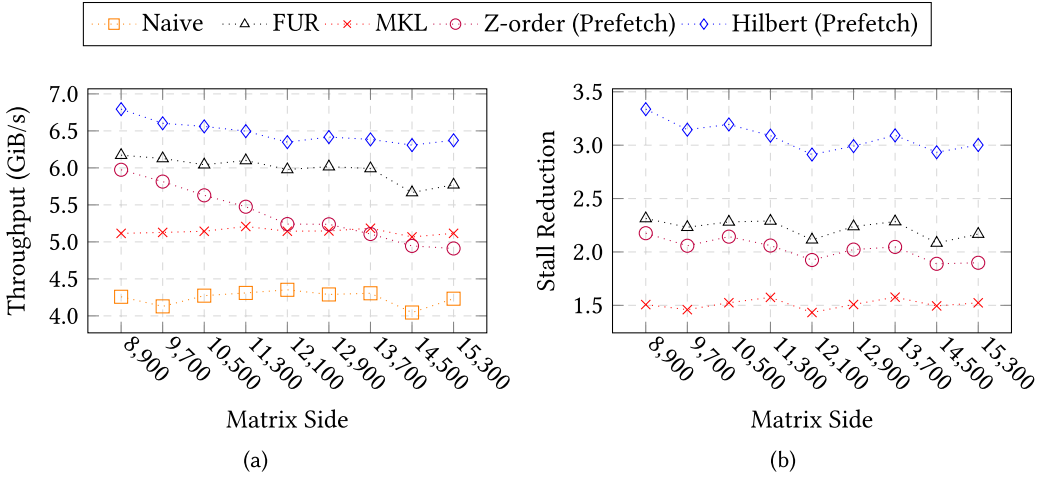


Fig. 11. (a) throughput in GiB/s and (b) memory-related stall reduction, in comparison to the naive approach, of the different out-of-place matrix transposition strategies for good non-square matrix sizes.

outperform both of our approaches due to the use of nano-programs, which results in smaller sequences of vertical scans, thus reducing the number of conflict misses. Nevertheless, FUR-Hilbert presented a smaller average throughput, 5.31 GiB/s, than the remaining cache-oblivious strategies. MKL presented the lowest performance gain with an average throughput of 5.07 GiB/s and the lowest stall reduction average of 1.45 $\times$.

5.2.8 Evaluation: Good Rectangular Matrices. Figures 11(a) and 11(b) depict the throughput and memory-related stalls reduction of the tested approaches for good-sized non-square rectangular matrices. The results obtained for this type of matrices are almost identical to the ones obtained for good-sized square matrices. It is possible to conclude that the throughput of these approaches is mostly oblivious to the input matrix shape. Our Hilbert blocking scheme, extended with standard prefetching techniques, presented the best throughput with an average of 6.47 GiB/s (an efficiency

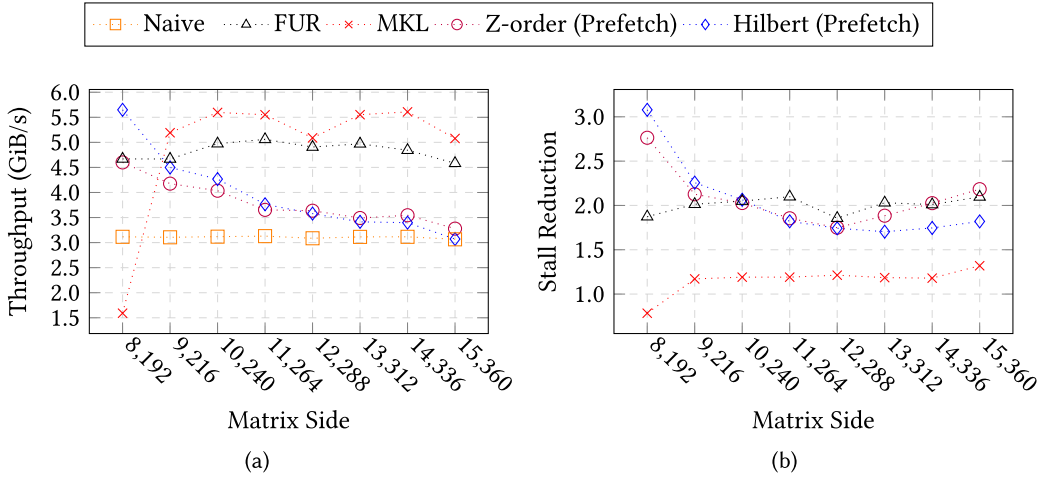


Fig. 12. (a) throughput in GiB/s and (b) memory-related stall reduction, in comparison to the naive approach, of the different out-of-place matrix transposition strategies for ugly rectangular matrix sizes.

of 61%) and the largest stall reduction of 3.08 $\times$. The application of software prefetching to our Hilbert blocking scheme resulted in an average throughput increase of 19% when compared to its non-prefetching counterpart. Our approach is closely followed by FUR-Hilbert with an average throughput of 5.98 GiB/s and an average stall reduction of 2.22 $\times$. None of the remaining approaches presents competitive results against our Hilbert-based blocking scheme or FUR-Hilbert.

5.2.9 Evaluation: Ugly Rectangular Matrices. Figures 12(a) and 12(b) depict the throughput and memory-related stalls reduction of the tested approaches for ugly-sized non-square rectangular matrices. In contrast to the evaluation of good-sized matrices, ugly-sized matrices showed drastic changes in throughput between squared and non-squared input matrices. For this type of matrices, MKL consistently outperforms all other approaches presenting an average throughput of 4.91 GiB/s (an efficiency of 46%). Unfortunately, the stall reduction presented by this approach does not seem to be correlated with its throughput. FUR-Hilbert is the only cache-oblivious approach that was not severely affected by the different matrix shapes, presenting competitive results against Intel MKL solution. This strategy presents an average throughput of 4.83 GiB/s and an average stall reduction of 2.02 $\times$. Both of our approaches presented poor performance, to the extent that the application of standard prefetching techniques resulted in performance decay. Our Hilbert and Z-order blocking schemes, extended with software prefetching, presented an overhead of 9% and 2% when compared to their non-prefetching counterparts. It seems that for ugly rectangular matrices, the use of software prefetching is causing the eviction of useful data in favor of data that is not currently needed. It is not yet clear why this phenomenon only occurs for ugly-sized non-squared matrices.

5.2.10 SIMD Hilbert Generator vs. Non-recursive Lindenmayer-System. Figures 13(a) and 13(b) depict the throughput, for good and ugly square matrices, of our blocking scheme for Hilbert curves generated by our SIMD Hilbert curve generator and the non-recursive Lindenmayer-System presented by Böhm et al. [5]. Both approaches were extended with software prefetching instructions. These approaches present identical behavior for both types of matrices. The non-recursive Lindenmayer-System is on average 2% and less than 1% slower than the same blocking scheme using our generator, for good and ugly matrices, respectively.

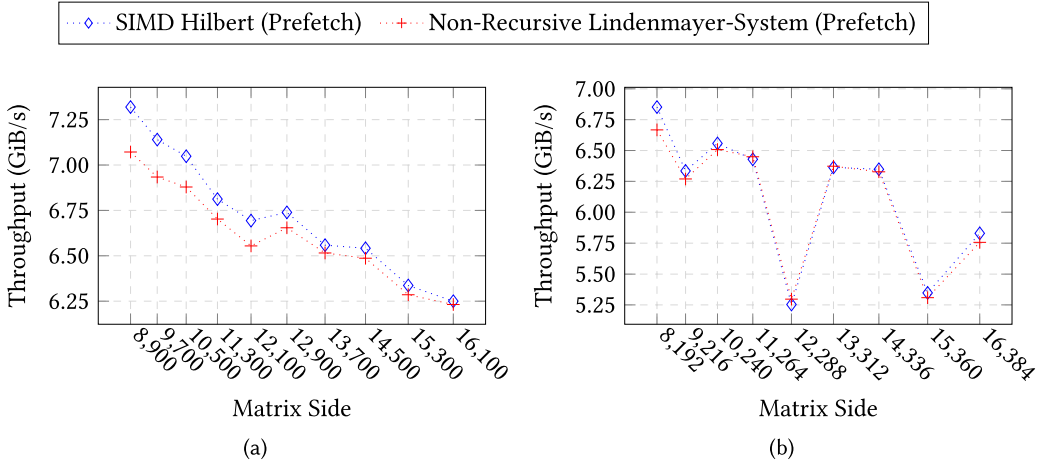


Fig. 13. Throughput in GiB/s of our blocking scheme by employing SIMD Hilbert Space-filling Curve Generator and the non-recursive Lindenmayer-System for (a) good square matrix sizes and (b) ugly square matrix sizes.

It becomes clear that the approach used to generate the Hilbert space-filling curve does not significantly impact the performance of our blocking scheme. Recall that our SIMD Hilbert Space-filling Curve Generator is on average $27.63\times$ faster than the non-recursive Lindenmayer-System, when optimized with compiler flag `-O3`.

It is also important to note that even though our generator requires an auxiliary buffer and, consequently, more memory movements, it still presents the best performance for this memory-bound problem. However, if there are memory constraints, then the non-recursive Lindenmayer-System version of our blocking scheme might be the most suitable approach.

6 FINAL REMARKS

In this work, we introduced a high-performance cache-oblivious algorithm for out-of-place transposition of rectangular matrices. Our blocking scheme leverages the locality preservation properties of space-filling curves, able to fill $2^l \times 2^l$ square grids, and software prefetching techniques, to improve the execution time of this problem in a portable manner. Our experiments show that optimized cache-oblivious solutions are able outperform the current hand-tuned cache-aware state-of-the-art solutions for the out-of-place rectangular transposition problem.

These results were only achieved due to the superior cache-behavior of our algorithm. Unfortunately, the results presented by the memory metrics of Section 5.2.4 are not always reflected in the behavior of the different approaches. This issue arises from the ambiguous definition of an execution stall in modern CPUs. Contemporary processors are able to issue multiple micro-instructions per cycle, making it unclear whether a stalled cycle refers to cycles where no micro-instruction was issued, or to cycles where the number of dispatched micro-instructions is below the maximum capabilities of a processing unit. The use of software prefetching techniques furthers the complexity of evaluating the different approaches, due to non-temporal prefetch instructions that bypass the cache and are not counted by cache-related PMUs.

The speedup of our blocking scheme suggests that our algorithm would perform well in a shared-memory parallel environment. In this environment, the costs associated with the curve generation can probably be amortized by computing a smaller curve that would be re-used by each processing unit. Finally, it is important to note that our generator and blocking scheme opened doors for other

high-performance cache-oblivious solutions, such as general dense linear algebra algorithms that inherently present poor memory-locality and can benefit from software prefetching.

Despite the fact that our SIMD Hilbert Space-filling Generator did not present significant performance gains in the context of matrix transposition, it still presented an outstanding speedup when compared to the remaining known methods for enumerating the points of a Hilbert space-filling curve.

ACKNOWLEDGMENTS

We would like to express gratitude to professor Nuno Roma and professor Pedro Tomás, who are members of the HPCAS research group at INESC-ID, for borrowing the machines used in the preliminary stages of this study.

REFERENCES

- [1] Guilherme Arroz, José Monteiro, and Arlindo Oliveira. 2018. *Computer Architecture*. World Scientific. DOI : <https://doi.org/10.1142/10940>
- [2] Michael Bader. 2012. *Space-filling Curves: An Introduction with Applications in Scientific Computing*. Vol. 9. Springer Science & Business Media.
- [3] Michael Bader and Christoph Zenger. 2006. Cache oblivious matrix multiplication using an element ordering based on a Peano curve. *Lin. Algeb. Appl.* 417, 2–3 (2006), 301–313.
- [4] J. Baert. 2018. Libmorton: C++ Morton encoding/decoding library. Retrieved from <https://github.com/Forceflow/libmorton>.
- [5] Christian Böhm, Martin Perdacher, and Claudia Plant. 2016. Cache-oblivious loops based on a novel space-filling curve. In *IEEE International Conference on Big Data (Big Data)*. IEEE. DOI : <https://doi.org/10.1109/bigdata.2016.7840585>
- [6] A. R. Butz. 1971. Alternative algorithm for Hilbert’s space-filling curve. *IEEE Trans. Comput.* C-20 (1971), 424–426.
- [7] S. Chatterjee and S. Sen. 2000. Cache-efficient matrix transposition. In *6th International Symposium on High-Performance Computer Architecture*. 195–205. DOI : <https://doi.org/10.1109/HPCA.2000.824350>
- [8] Ningtao Chen, Nengchao Wang, and Baochang Shi. 2007. A new algorithm for encoding and decoding the Hilbert order. *Softw.: Pract. Exper.* 37, 8 (2007), 897–908. DOI : <https://doi.org/10.1002/spe.793>
- [9] N. Chomsky. 1956. Three models for the description of language. *IRE Trans. Inf. Theor.* 2, 3 (1956), 113–124.
- [10] Agner Fog. 2020. Optimizing software in C++. Retrieved from https://www.agner.org/optimize/optimizing_cpp.pdf.
- [11] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. 2012. Cache-oblivious algorithms. *ACM Trans. Algor.* 8, 1 (Jan. 2012), 1–22. DOI : <https://doi.org/10.1145/2071379.2071383>
- [12] Kazushige Goto. 2007. GotoBLAS. Retrieved from <http://www.tacc.utexas.edu/resources/software/>.
- [13] David Hilbert. 1935. Über die stetige Abbildung einer Linie auf ein Flächenstück. In *Dritter Band: Analysis: Grundlagen der Mathematik: Physik Verschiedenes*. Springer, 1–2.
- [14] Intel® Intrinsics Guide. Intel® Intrinsics Guide. Retrieved from <https://software.intel.com/sites/landingpage/IntrinsicsGuide/>.
- [15] Guohua Jin and John Mellor-Crummey. 2005. Space-filling curve generation: A table-based approach. *International Conference on Algorithmic Mathematics and Computer Science*. 40–46.
- [16] K. Patrick Lorton and David S. Wise. 2007. Analyzing block locality in Morton-order and Morton-hybrid matrices. *ACM SIGARCH Comput. Archit. News* 35, 4 (2007), 6–12.
- [17] John D. McCalpin. 1991–2007. *STREAM: Sustainable Memory Bandwidth in High Performance Computers*. Technical Report. University of Virginia, Charlottesville, Virginia. Retrieved from <http://www.cs.virginia.edu/stream/>.
- [18] MKL 2020. Intel® Math Kernel Library. Retrieved from <https://software.intel.com/content/www/us/en/develop/tools/math-kernel-library.html>.
- [19] Michael Mol. 2007. Matrix transposition. Retrieved from https://rosettacode.org/wiki/Matrix_transposition#C.
- [20] G. M. Morton. 1966. A computer oriented geodetic data base and a new technique in file sequencing (Ottawa, Canada: IBM). (1966).
- [21] GNU. 2013. Other Built-in Functions Provided by GCC. Retrieved from <https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html>.
- [22] Przemysław Prusinkiewicz and James Hanan. 1989. *Lindenmayer Systems, Fractals, and Plants*. Springer New York. DOI : <https://doi.org/10.1007/978-1-4757-1428-9>

- [23] Andrey Vladimirov. 2013. Multithreaded transposition of square matrices with common code for Intel Xeon processors and Intel Xeon Phi coprocessors. https://www.researchgate.net/publication/258048110_Multithreaded_Transposition_of_Square_Matrices_with_Common_Code_for_Intel_Xeon_Processors_and_Intel_Xeon_Phi_Coprocessors.
- [24] Zhang Xianyi, Wang Qian, and Zaheer Chothia. 2012. OpenBLAS. Retrieved from <http://xianyi.github.io/OpenBLAS>.

Received 7 January 2021; revised 25 May 2022; accepted 27 June 2022